



DEPARTMENT OF
COMPUTER SCIENCE

Spring 2026

Master of Science in Cyber Security Project Report

Computer Science Department

California State University, **Dominguez Hills**

Security Visibility Tradeoffs in TLS-Encrypted Cloud Network Environments

A Project

Presented

to the faculty of

California State University, Dominguez Hills

In Partial Fulfillment

of the Requirements for the Degree

Master of Science

in

Cyber Security

by

Michael Fisher

Spring 2026

I would like to dedicate this project to both my parents and my sister. Without their continuous support and inspiration, I would not have been able to persevere and achieve my academic goals.

ACKNOWLEDGEMENTS

Special thanks to Dr. Brighu Celly, Dr. Alireza Izaddoost, and Dr. Mohsen Beheshti for their valuable insight and guidance during my time as a student at California State University, Dominguez Hills.

Authors Declaration

I hereby declare that this project consists of original work which I have authored. This is a true copy of the work, including any required final revisions, as accepted by my committee or course instructor.

Name	Michael Fisher
Signature	
Date	05/02/2026

TABLE OF CONTENTS

DEDICATION.....	ii
ACKNOWLEDGEMENTS.....	iii
AUTHOR’S DECLARATION.....	iv
TABLE OF CONTENTS.....	v
LIST OF FIGURES.....	vi
ABSTRACT.....	vii
CHAPTER	
1. INTRODUCTION.....	1
2. BACKGROUND.....	3
3. PROPOSED METHOD.....	7
4. IMPLEMENTATION.....	11
EC2 Setup.....	12
Security groups.....	13
Juice Shop Deployment.....	14
Suricata.....	15
Zeek.....	16
Nginx HTTPS Setup.....	17
5. RESULTS & DISCUSSION	19
6. CONCLUSION AND FUTURE WORK.....	26
7. CONTRIBUTION.....	27
8. REFERENCES.....	28

LIST OF FIGURES

	PAGE
1. TLS protocol key negotiation process from Zheng et al. [2].....	4
2. Experimental Design and Data Flow Architecture for HTTP and HTTPS.....	8
3. AWS EC2 Instance provisioned	9
4. EC2 security group inbound rules configured for controlled HTTP, HTTPS, and SSH access.....	13
5. OWASP Juice Shop successfully deployed on the target EC2 instance over HTTP.....	14
6. Suricata installation, rule configuration, and engine initialization on the target EC2 instance.....	15
7. Zeek installation and system configuration process during package setup.....	16
8. Nginx configuration and TLS certificate setup for enabling HTTPS.....	17
9. OWASP Juice Shop successfully accessed over HTTPS using a self-signed certificate.....	18
10. HTTP SQL Injection (Zeek http.log).....	20
11. HTTPS SQL Injection Comparison of Burp Suite decrypted HTTPS request and Zeek TLS metadata logs during SQL Injection testing.....	21
12. HTTP vs HTTPS Visibility Comparison Table	23

ABSTRACT

This study evaluates how Transport Layer Security (TLS) encryption impacts the visibility and detection capabilities of network-based intrusion detection systems (IDS) in a cloud environment. A controlled experimental lab was implemented using Amazon Web Services (AWS), where the OWASP Juice Shop application was deployed and monitored using Suricata and Zeek. Application-layer attacks, including SQL injection, cross-site scripting (XSS), and repeated failed login attempts, were executed under both HTTP and HTTPS conditions to compare detection behavior and visibility differences.

The results demonstrate that while HTTP traffic provides full application-layer visibility, HTTPS significantly reduces this visibility by encrypting request contents. Under HTTPS, detection shifts from payload inspection to analysis of TLS metadata, connection behavior, and traffic patterns. Suricata's signature-based detection was limited under encryption, while Zeek remained effective by providing connection-level and TLS metadata. These findings confirm that TLS introduces a fundamental visibility tradeoff, requiring layered detection strategies to maintain security in encrypted environments.

CHAPTER 1

INTRODUCTION

Over the past decade, the widespread adoption of Transport Layer Security (TLS) has fundamentally transformed how data is transmitted across modern networks. Encryption has become the default standard for web applications and cloud-hosted services, driven by the need to protect sensitive data, ensure confidentiality, and maintain trust in digital systems. While this shift strengthens security from a privacy standpoint, it simultaneously introduces new challenges for network monitoring and threat detection.

Network-based intrusion detection systems (IDS), such as Suricata and Zeek, have traditionally relied on deep packet inspection (DPI) techniques to analyze network traffic and identify malicious behavior. These systems examine packet payloads to detect known attack signatures, enabling identification of threats such as SQL injection, cross-site scripting (XSS), and other application-layer attacks. However, as encrypted traffic becomes dominant, the effectiveness of DPI-based detection is significantly reduced. Without access to payload content, IDS tools can no longer directly inspect or analyze the data required to identify many types of attacks.

This limitation presents a critical challenge in modern cloud environments, where encrypted traffic is pervasive and centralized monitoring remains essential. Although encryption prevents direct inspection of application-layer data, it does not eliminate visibility entirely. Metadata such as TLS handshake information, connection patterns, and traffic behavior remains observable. The extent to which this remaining data can support meaningful detection, however, remains a key question in evaluating IDS effectiveness.

This project addresses the following research question:

How does TLS encryption impact the visibility and detection capabilities of network-based intrusion detection systems in cloud environments?

The study is based on the hypothesis that network-based IDS experience a measurable reduction in detection effectiveness when payload inspection is removed by TLS encryption, while still retaining limited visibility through handshake metadata and flow-level features.

To evaluate this, a controlled cloud-based lab environment was developed using Amazon Web Services (AWS). A vulnerable web application, OWASP Juice Shop, was deployed and monitored using Suricata and Zeek. A series of application-layer attacks, including SQL injection, cross-site scripting, and repeated login attempts, were executed over both HTTP and HTTPS. By keeping all variables constant except for the presence of TLS encryption, the experiment provides a direct comparison of IDS behavior in encrypted versus unencrypted conditions.

The significance of this study lies in its demonstration of a fundamental tradeoff in modern cybersecurity. While TLS encryption enhances privacy and data protection, it simultaneously reduces the effectiveness of traditional network-based detection techniques. Prior research has shown that intrusion detection systems experience reduced performance in encrypted environments, particularly when analyzing application-layer attacks in cloud-based systems [1]. This project builds upon these findings by providing a practical, experimental evaluation of how visibility loss impacts real-world IDS tools. The results highlight the need for alternative detection strategies that rely on behavioral analysis and metadata rather than direct payload inspection.

CHAPTER 2

BACKGROUND

As discussed in Chapter 1, the widespread adoption of TLS encryption has significantly changed the operating conditions of network-based intrusion detection systems. While encryption improves confidentiality and protects sensitive communications, it also reduces the visibility that traditional monitoring tools rely on to inspect network traffic. This challenge is especially important in cloud environments, where encrypted traffic is now the norm and defenders must operate with limited access to application-layer content.

TLS encryption secures communications between clients and servers by protecting data in transit from interception and tampering. However, this protection introduces an important tradeoff for network defenders. Once traffic is encrypted, the application-layer payload is no longer directly visible to monitoring tools, limiting the ability to inspect request bodies, headers, parameters, and other content-based indicators of malicious activity. Zheng et al. [2] explain that traditional deep packet inspection methods become ineffective in TLS environments because encrypted payloads no longer contain readable content that can be matched through signature inspection.

Although payload data becomes hidden, not all traffic characteristics disappear. Certain elements of the TLS handshake remain visible, including protocol negotiation, session metadata, and cryptographic parameters. This remaining visibility is significant because it provides one of the only ways for network defenders to analyze encrypted sessions without performing decryption. The distinction between encrypted payload content and observable metadata is central to the present study.

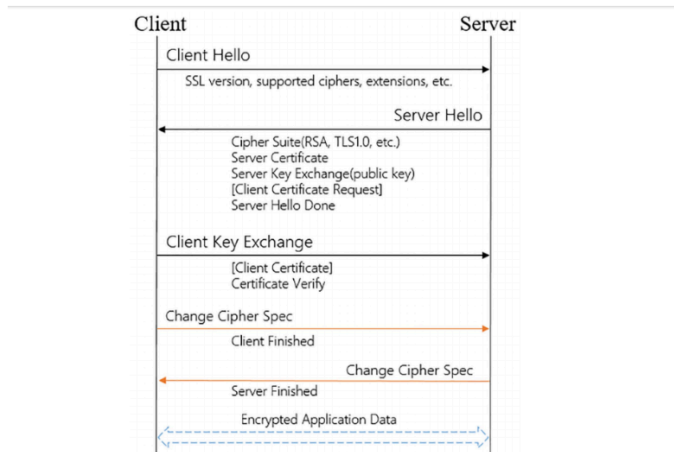


Figure 2.1: TLS protocol key negotiation process from Zheng et al. [2]

Network-based intrusion detection systems are designed to monitor traffic for signs of malicious behavior, policy violations, and abnormal communication patterns. Traditionally, these systems depend on access to packet contents to detect threats with a high degree of precision. When traffic is unencrypted, IDS tools can directly examine payloads and identify recognizable attack indicators such as malicious scripts, suspicious parameters, and exploit strings.

In cloud-based environments, however, the effectiveness of these systems becomes constrained. The increasing prevalence of encryption limits access to the traffic contents that traditional IDS techniques were designed to analyze. Qutqut et al. [1] demonstrate that IDS performance declines in cloud environments when visibility is restricted, particularly when encrypted traffic removes access to application-layer content. This reinforces the need to evaluate not only whether traffic is observable, but what type of information remains available for detection.

Enterprise network security commonly relies on tools such as Suricata, Snort, Zeek, and integrated security platforms that combine IDS, network detection and response (NDR), and SIEM capabilities. These tools provide visibility into network behavior, but their effectiveness depends on the type of traffic being inspected. When traffic is unencrypted, signature-based tools can inspect payloads directly. When traffic is encrypted, defenders must rely more heavily on metadata, connection behavior, TLS fingerprints, endpoint telemetry, proxy logs, and application-layer logging.

Suricata is widely used as a signature-based intrusion detection system. Its strength lies in its ability to inspect traffic contents and compare them against known detection rules. In unencrypted environments, this enables Suricata to identify application-layer threats such as SQL injection and cross-site scripting with considerable precision.

The limitation of this approach becomes clear under TLS encryption. Because signature-based detection depends on payload visibility, HTTPS removes much of the information required for accurate application-layer detection. As Zheng et al. [2] note, encrypted traffic cannot be effectively analyzed through traditional deep packet inspection because the relevant content is hidden. As a result, Suricata may still observe connections and general anomalies, but its ability to detect specific content-driven attacks is significantly reduced.

In contrast, Zeek is designed to emphasize connection metadata, protocol behavior, and flow-level analysis. Rather than relying solely on packet contents, Zeek records observable traffic features such as session timing, source and destination hosts, ports, protocol usage, and TLS handshake metadata. This makes Zeek particularly valuable in environments where payload inspection is limited or unavailable.

The usefulness of metadata-driven analysis in encrypted environments is supported by both academic and industry research. Zheng et al. [2] highlight the detection value of TLS handshake features and statistical characteristics, while Cisco's analysis of encrypted malware traffic demonstrates that metadata derived from handshake messages and packet timing can support detection without requiring decryption [3]. These approaches do not reveal the exact content of a session, but they enable analysts to make informed inferences about traffic behavior.

Prior literature consistently shows that the transition from HTTP to HTTPS changes both what can be observed and how detection must be performed. In HTTP environments, monitoring tools can inspect the complete application-layer exchange. In HTTPS environments, visibility is reduced to metadata, handshake information, and flow characteristics unless decryption is introduced.

Outqut et al. [1] show that intrusion detection systems lose effectiveness in encrypted cloud environments when application-layer visibility is removed. Zheng et al. [2] similarly

demonstrate that meaningful detection is still possible through handshake and statistical features, but this represents a shift away from traditional payload inspection. Together, these studies support the central premise of this project: TLS encryption does not eliminate visibility, but it fundamentally changes the type of visibility available to network defenders.

Although prior research establishes that encrypted traffic reduces the effectiveness of payload-based inspection, a practical gap remains. Many studies focus on classification models, feature engineering, or large-scale datasets, but fewer provide controlled comparisons of IDS behavior across HTTP and HTTPS using identical attack scenarios.

This project addresses that gap by implementing a controlled AWS-based lab environment and executing the same application-layer attacks over both unencrypted and encrypted traffic. By isolating TLS as the primary experimental variable, the study provides a practical demonstration of how visibility changes affect Suricata and Zeek in real monitoring conditions. In doing so, the project connects theoretical research to applied network defense analysis and clarifies the tradeoffs introduced by encrypted traffic.

CHAPTER 3

PROPOSED METHOD

The purpose of this project is to evaluate how Transport Layer Security (TLS) encryption impacts the visibility and detection capabilities of network-based intrusion detection systems in a cloud environment. Specifically, this study examines how the transition from unencrypted HTTP traffic to encrypted HTTPS traffic affects the ability of intrusion detection tools to observe, analyze, and identify application-layer attacks.

To achieve this objective, a controlled experimental methodology was designed in which identical attack scenarios were executed under both HTTP and HTTPS conditions. By maintaining a consistent environment and varying only the presence of TLS encryption, the study isolates encryption as the primary factor influencing detection outcomes. This approach enables a direct comparison of how visibility and detection behavior change across both conditions.

The research follows a controlled experimental design. The same vulnerable application, attack techniques, and monitoring tools were used across both test conditions, with TLS encryption serving as the sole independent variable. In the unencrypted scenario, network traffic was transmitted using HTTP, allowing full visibility into application-layer data. In the encrypted scenario, HTTPS was used to secure communications through TLS, preventing direct inspection of payload content. This design ensures that any observed differences in intrusion detection performance can be attributed to the presence of encryption rather than external variables.

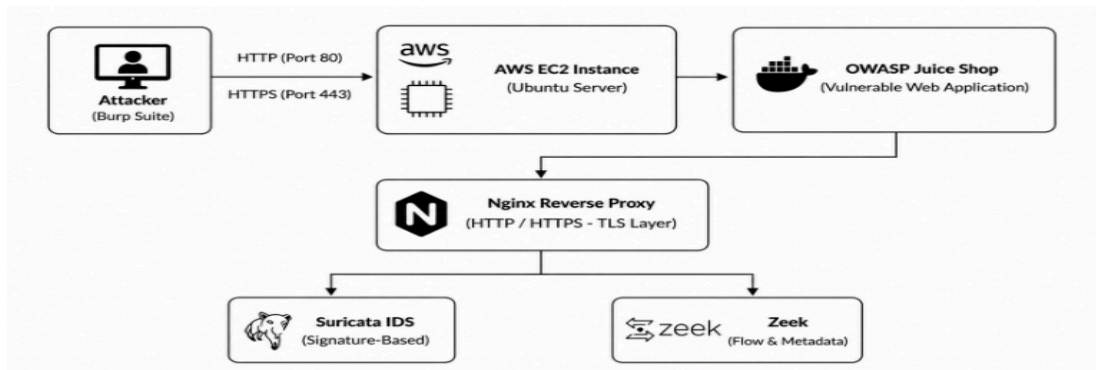


Figure 3.1: Experimental Design and Data Flow Architecture for HTTP and HTTPS

The experimental environment was deployed in Amazon Web Services (AWS) using an EC2 instance running Ubuntu Server. The target system hosted the OWASP Juice Shop application within a Docker container, providing a controlled platform for executing common web application attacks. To support both HTTP and HTTPS testing, Nginx was configured as a reverse proxy using a self-signed TLS certificate. This configuration allowed the same application to be accessed over both protocols without modifying the underlying system. Intrusion detection was performed using Suricata and Zeek, which were installed and configured on the instance. Suricata was used for signature-based traffic inspection, while Zeek was used to capture connection-level metadata and behavioral patterns.

Attack traffic was generated from a local workstation using Burp Suite Community Edition. Burp Suite enabled interception, modification, and forwarding of HTTP requests, ensuring consistent attack execution across both HTTP and HTTPS environments.

The primary independent variable in this study is the presence of TLS encryption, which distinguishes the two testing conditions: HTTP (unencrypted) and HTTPS (encrypted). The dependent variables include intrusion detection system alert generation, visibility of application-layer data, availability of TLS metadata, and observable network behavior. These variables were evaluated by comparing the outputs of Suricata and Zeek across both conditions.

Measurement criteria were based on whether attacks were detected, what information was visible within logs, and how each tool represented network activity under different levels of visibility. This approach supports a qualitative and comparative analysis of detection capabilities rather than relying solely on binary detection outcomes.

Three types of application-layer attacks were selected for this study: SQL injection, cross-site scripting (XSS), and repeated failed login attempts. These attacks were chosen because they represent common web-based threats that typically rely on payload inspection for detection. Each attack was executed twice—once over HTTP and once over HTTPS—using identical payloads and delivery methods. Burp Suite was used to intercept and modify requests before forwarding them to the target application, ensuring that the attack structure remained consistent across both conditions.

Data collection was performed using both Suricata and Zeek during each attack scenario. Suricata logs, including *fast.log* and *eve.json*, were monitored to capture alerts and network events. Zeek logs, including *conn.log*, *http.log*, and *ssl.log*, were used to observe connection behavior, application-layer activity, and TLS metadata. All data was collected in real time during attack execution and preserved for analysis.

The analysis focused on comparing the outputs of Suricata and Zeek across both HTTP and HTTPS environments. This included identifying differences in detection capability, evaluating changes in log visibility, and examining how each tool represented network activity under encryption. Rather than relying solely on detection success, the analysis emphasizes the type and quality of data available to each system and how encryption alters overall network visibility.

All testing conducted in this study was performed within a controlled and isolated cloud environment. The target application, OWASP Juice Shop, is intentionally vulnerable and designed for security testing purposes. No external systems or unauthorized networks were targeted during the experiment. The use of Burp Suite and attack techniques was strictly limited to the lab environment, ensuring that all activities were conducted ethically and in accordance with responsible cybersecurity practices.

This methodology provides a structured framework for evaluating how TLS encryption impacts intrusion detection visibility and performance. The next chapter presents the implementation of the experimental environment.

CHAPTER 4

IMPLEMENTATION

This chapter describes the implementation of the cloud-based lab environment used to evaluate how TLS encryption affects network intrusion detection visibility. The purpose of this phase was to build a controlled environment in which the same vulnerable application could be observed under both HTTP and HTTPS conditions while maintaining consistent monitoring and attack generation procedures. The environment was designed to isolate TLS encryption as the primary experimental variable.

The experimental environment was deployed using an Amazon Web Services (AWS) EC2 instance running Ubuntu Server. AWS was selected because it provides a scalable and realistic cloud platform that allows external access while maintaining full control over configuration. This ensured that the vulnerable application and monitoring tools could be tested in conditions similar to real-world deployments.

During provisioning, the EC2 instance was configured within a virtual private cloud (VPC) and associated subnet, enabling internal system communication and external connectivity through a public interface. The instance type (t3.small) was selected to balance performance and cost, although this choice later introduced minor resource constraints during simultaneous IDS operation. Establishing a stable and reachable host system was essential, as all subsequent components depended on this infrastructure.

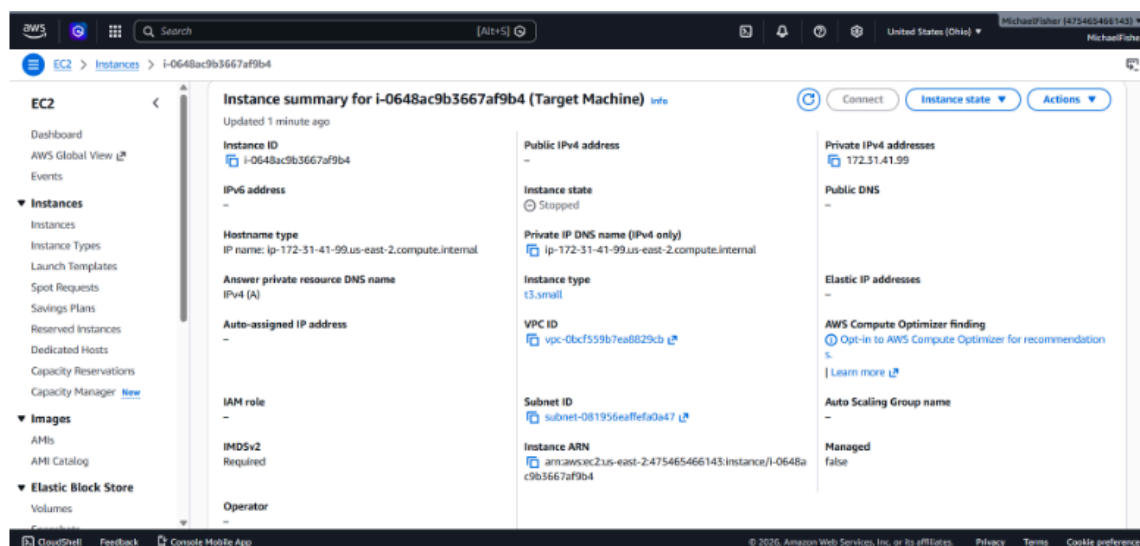


Figure 4.1: AWS EC2 Instance provisioned for the experimental lab environment

As shown in Figure 4.1, the EC2 instance was successfully configured with the required network and system settings, confirming readiness for application deployment and remote access.

Following instance provisioning, network access controls were configured using AWS security groups to define permitted traffic. Inbound rules were established for SSH (port 22), HTTP (port 80), and HTTPS (port 443). These configurations ensured that the application could be accessed from the attacker workstation while allowing network monitoring tools to observe incoming traffic. Restricting traffic to only required ports also maintained a controlled testing environment, ensuring that all observed activity was directly related to the experiment.

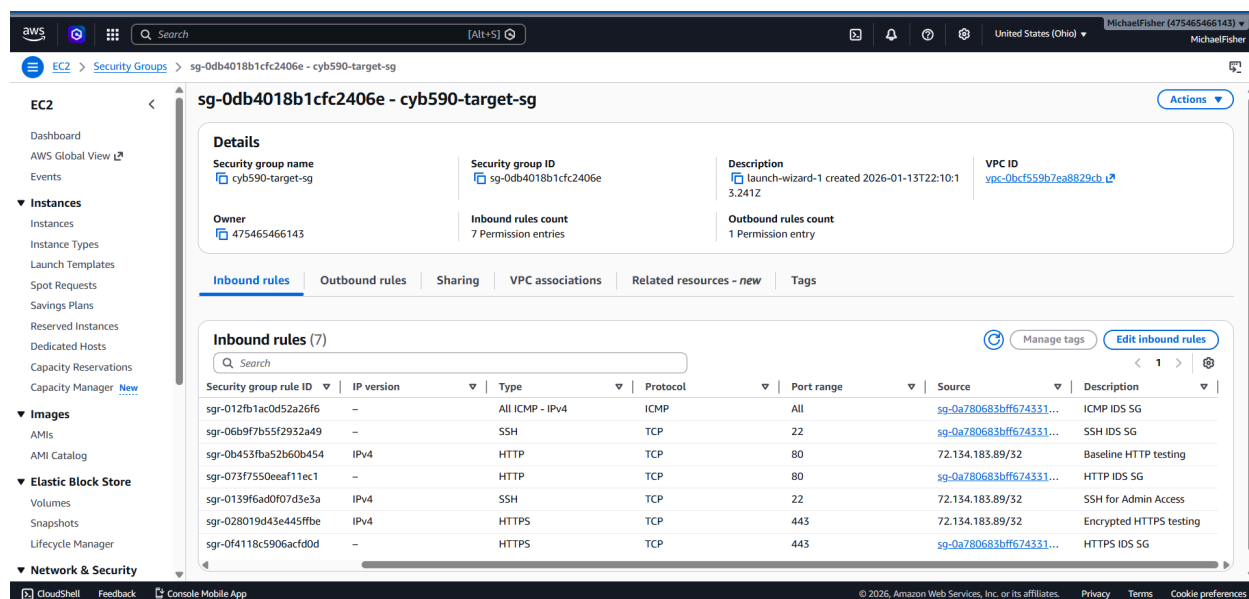


Figure 4.2: EC2 security group inbound rules configured for controlled HTTP, HTTPS, and SSH access

As figure 4.2 illustrates the configured inbound rules, showing that only specific ports were opened for testing purposes. This configuration enabled controlled communication between the attacker and the target system while ensuring that the intrusion detection tools could monitor relevant traffic flows.

With network access established, OWASP Juice Shop was deployed as the target application. Juice Shop was selected because it is intentionally vulnerable and designed for web application security testing, making it suitable for controlled execution of attacks such as SQL injection, cross-site scripting, and login abuse. The application was deployed using Docker to simplify installation and ensure consistency without modifying the underlying operating system.

The initial deployment was performed over HTTP to establish a baseline environment. This step ensured that the application, browser access, and system configuration were functioning correctly before introducing encryption. Establishing this baseline was critical for later comparison with HTTPS behavior.

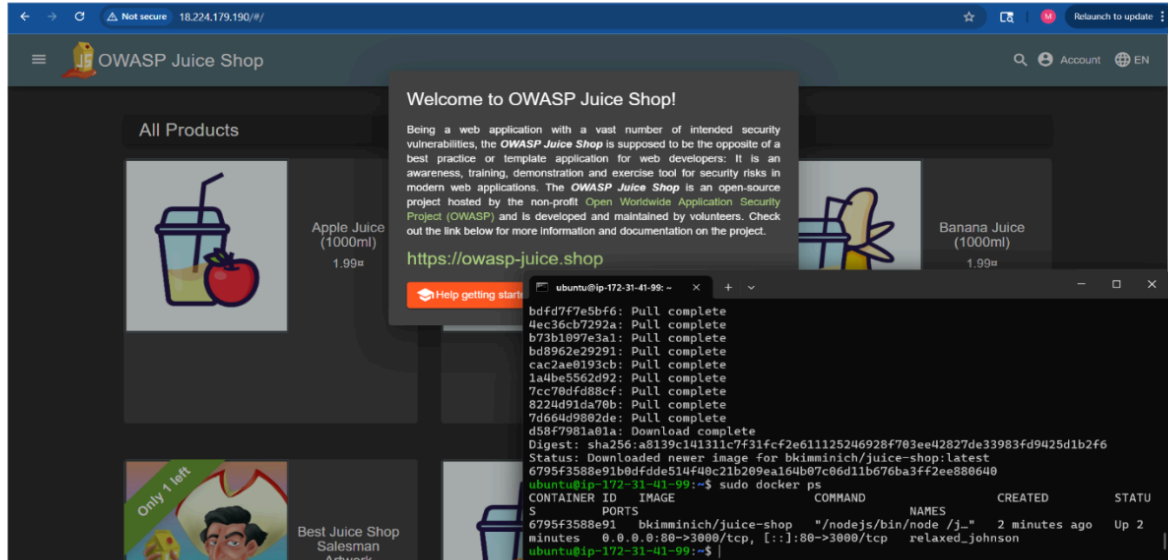


Figure 4.3: OWASP Juice Shop successfully deployed on the target EC2 instance over HTTP

Once the application was accessible, Burp Suite Community Edition was configured on the local workstation to enable controlled attack generation. Burp Suite was used as an intercepting proxy, allowing HTTP requests to be captured, modified, and forwarded to the target application. This enabled consistent execution of identical attack payloads under both HTTP and HTTPS conditions.

Suricata was then installed and configured to provide signature-based intrusion detection. Its role in the experiment was to analyze network traffic and generate alerts when known patterns or anomalies were detected. Suricata was particularly relevant for the HTTP phase, where application-layer payloads were visible.

```

uricata.rules: total: 65365; enabled: 49501; added: 0; removed 0; modified:
0
10/4/2026 -- 21:08:20 - <Info> -- Writing /var/lib/suricata/rules/classifica
tion.config
10/4/2026 -- 21:08:20 - <Info> -- No changes detected, exiting.
ubuntu@ip-172-31-41-99:~$ sudo suricata -c /etc/suricata/suricata.yaml -i en
s5
10/4/2026 -- 21:08:27 - <Notice> - This is Suricata version 6.0.4 RELEASE ru
nning in SYSTEM mode
10/4/2026 -- 21:08:27 - <Warning> - [ERRCODE: SC_ERR_NO_RULES(42)] - No rule
files match the pattern /etc/suricata/rules/suricata.rules
10/4/2026 -- 21:08:27 - <Warning> - [ERRCODE: SC_ERR_NO_RULES_LOADED(43)] -
1 rule files specified, but no rules were loaded!
10/4/2026 -- 21:08:27 - <Notice> - all 2 packet processing threads, 4 manage
ment threads initialized, engine started.
^C10/4/2026 -- 21:11:27 - <Notice> - Signal Received. Stopping engine.
10/4/2026 -- 21:11:28 - <Notice> - Stats for 'ens5': pkts: 60, drop: 0 (0.0
0%), invalid checksum: 0
ubuntu@ip-172-31-41-99:~$ sudo nano /etc/suricata/suricata.yaml
ubuntu@ip-172-31-41-99:~$ sudo suricata -c /etc/suricata/suricata.yaml -i en
s5
10/4/2026 -- 21:16:44 - <Notice> - This is Suricata version 6.0.4 RELEASE ru
nning in SYSTEM mode
10/4/2026 -- 21:18:35 - <Notice> - all 2 packet processing threads, 4 manage
ment threads initialized, engine started.
^C10/4/2026 -- 21:19:29 - <Notice> - Signal Received. Stopping engine.
10/4/2026 -- 21:19:30 - <Notice> - Stats for 'ens5': pkts: 23, drop: 0 (0.0
0%), invalid checksum: 0
ubuntu@ip-172-31-41-99:~$ grep default-rule-path /etc/suricata/suricata.yaml
default-rule-path: /var/lib/suricata/rules
ubuntu@ip-172-31-41-99:~$ grep -A 5 rule-files /etc/suricata/suricata.yaml
rule-files:
- suricata.rules

##
## Auxiliary configuration files.
##
ubuntu@ip-172-31-41-99:~$ ls /var/lib/suricata/rules/
classification.config  suricata.rules
ubuntu@ip-172-31-41-99:~$ |

```

Figure 4.4: Suricata installation, rule configuration, and engine initialization on the target EC2 instance

As shown in Figure 4.4, Suricata was successfully configured, including rule loading and engine initialization. This confirmed that the IDS was prepared to monitor network traffic and evaluate requests against its detection rules.

Zeek was also installed to provide network visibility through connection-level and metadata-based analysis. Unlike Suricata, Zeek does not rely on signature matching, but instead records detailed logs of network behavior, including connection activity and TLS handshake information. This capability made Zeek particularly valuable for analyzing encrypted traffic. During installation, system dependencies and services were configured to ensure proper operation. Zeek was then configured to listen on the active network interface of the EC2 instance, enabling real-time traffic monitoring and log generation.

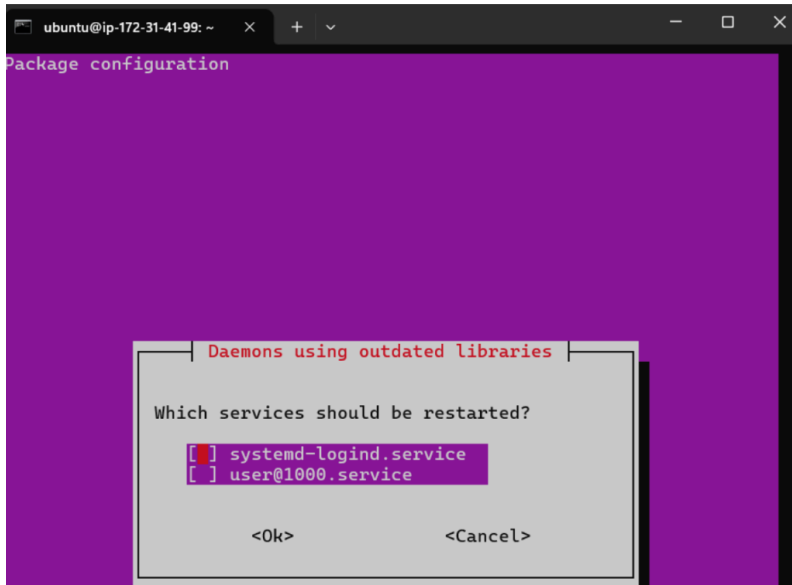


Figure 4.5: Zeek installation and system configuration process during package setup

Figure 4.5 shows the system-level configuration required to prepare Zeek for operation, confirming that the environment was ready to capture network metadata and behavioral information.

After establishing a stable HTTP baseline, the environment was extended to support HTTPS using Nginx as a reverse proxy with a self-signed TLS certificate. This approach allowed encryption to be introduced without modifying the underlying application, ensuring that the same system could be used for both testing conditions. Nginx handled TLS termination, allowing encrypted traffic to be received over port 443 and forwarded to the application. A self-signed certificate was used because the environment was controlled and did not require a trusted certificate authority.

CHAPTER 5

RESULTS AND DISCUSSION

The purpose of this chapter is to present and analyze the results of the HTTP and HTTPS testing phases conducted in the cloud lab environment. The analysis focuses on how TLS encryption affected the visibility and detection capabilities of the monitoring tools used in the project. While HTTPS preserves network communication, it removes direct application-layer payload visibility, requiring defenders to rely on metadata, TLS characteristics, flow behavior, and supporting data sources such as application and authentication logs.

The results are organized around the three attack scenarios tested during the experiment: SQL injection, cross-site scripting, and repeated failed login attempts. Each attack was executed under both HTTP and HTTPS conditions to compare differences in payload visibility, IDS alerting, TLS metadata, and flow-level behavior. This chapter also evaluates the different roles of Suricata and Zeek, with Suricata representing signature-based intrusion detection and Zeek representing metadata and behavioral network analysis.

The HTTP baseline phase provided the highest level of visibility because the traffic was not encrypted. This allowed network monitoring tools to observe application-layer communication between the client and the OWASP Juice Shop application. As a result, HTTP testing served as the control condition for evaluating how much visibility changed once HTTPS was enabled.

During the HTTP SQL injection test, Zeek's `http.log` captured application-layer request activity associated with the attack. The POST request to the authentication endpoint (`/rest/user/login`) was recorded along with request metadata such as source and destination IP addresses, request methods, and response status codes. Although Zeek does not explicitly label the request as a SQL injection attack, HTTP visibility allows analysts to correlate request behavior with malicious activity. In this scenario, the SQL injection payload was delivered through Burp Suite, resulting in a successful authentication bypass. The corresponding HTTP

During HTTPS testing, Burp Suite displayed decrypted HTTP requests because it operated as an intercepting proxy. This does not mean the network traffic was unencrypted. Burp was able to decrypt and display the application-layer request for testing purposes, while network-based monitoring tools observed the traffic as TLS/HTTPS traffic on port 443. This distinction is important because a passive network IDS does not automatically receive the decrypted payload.

The HTTPS phase revealed the central visibility tradeoff of the project. The same attacks could still be performed against the application, but the monitoring tools no longer had direct access to request payloads. Instead, the available evidence shifted toward encrypted session metadata and connection behavior.

During the HTTPS SQL injection test, the attack was successfully executed through Burp Suite, resulting in an authentication bypass similar to the HTTP phase. However, the key difference was in what network-based monitoring tools were able to observe. Burp Suite displayed the full HTTP request and SQL injection payload because it was positioned to decrypt TLS traffic. In contrast, Zeek did not capture the application-layer request in `http.log`. Instead, activity was recorded in `ssl.log` and `conn.log`, where only encrypted session metadata was visible.

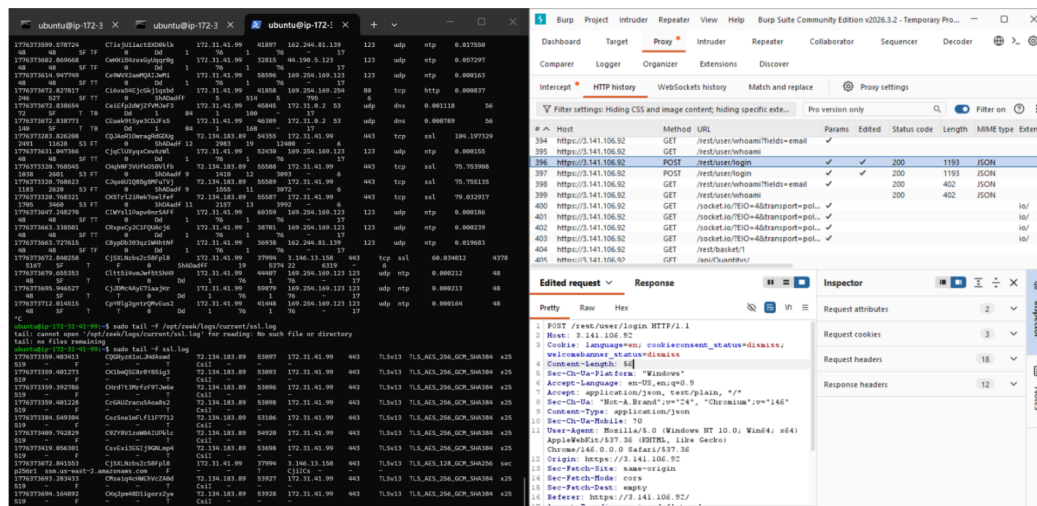


Figure 5.2 HTTPS SQL Injection Comparison of Burp Suite decrypted HTTPS request and Zeek TLS metadata logs during SQL Injection testing

The Zeek logs showed multiple TLS connections over port 443, including information such as TLS version, cipher suites, session identifiers, and connection timing. While this confirmed that communication between the client and server occurred, the SQL injection payload itself was not visible. This demonstrates that the attack remained functional, but network-based monitoring tools lost access to the application-layer contents. As a result, detection based on payload inspection was no longer possible, and visibility shifted toward metadata and behavioral indicators.

During HTTPS testing, XSS attempts were no longer visible at the application layer due to encryption. Zeek logs reflected encrypted session activity in `ssl.log` and connection-level behavior in `conn.log`. As a result, the injected script content could not be observed directly by the network monitoring tools. Detection in this context would require additional visibility sources, such as endpoint logging, application logs, browser security controls, web application firewall logs, or behavioral analysis.

Repeated failed login attempts over HTTPS were still observable as multiple connection attempts and session activity within `conn.log`. However, the contents of the login requests were not visible. This means analysts could observe repeated communication patterns but could not directly inspect the submitted username, password, or request body from network traffic alone. Analysts would need to rely on frequency, timing, source address behavior, session volume, and correlation with authentication logs to determine whether login activity was malicious.

Suricata was most effective when payload content was visible. During HTTP testing, Suricata was able to evaluate traffic using its rule set and identify suspicious or abnormal HTTP behavior when rule conditions were met. However, during HTTPS testing, the same application-layer payloads were encrypted, preventing Suricata from directly inspecting request bodies, parameters, and headers. As a result, Suricata's detection capability shifted away from application-layer signatures and toward more general TCP/TLS indicators. These findings align with prior research indicating that signature-based intrusion detection systems experience reduced effectiveness when encrypted traffic removes payload visibility [1].

In contrast to Suricata's reliance on payload inspection, Zeek provided consistent visibility across both HTTP and HTTPS conditions through structured logging of network

behavior. Under HTTP, Zeek's http.log provided application-layer details such as request methods, hosts, URIs, response codes, and user-agent information. Under HTTPS, Zeek primarily recorded encrypted session metadata in ssl.log and connection behavior in conn.log. Although Zeek did not identify the SQL injection payload directly under HTTPS, it preserved important investigative context such as source and destination addresses, port usage, session timing, TLS characteristics, and connection behavior. This supports research emphasizing the role of metadata and flow-based analysis in encrypted traffic environments, where payload visibility is limited but behavioral patterns remain accessible [4].

The comparison between HTTP and HTTPS testing highlights that the key difference is not whether network activity can be observed, but what level of detail remains accessible. Under HTTP, monitoring tools have direct access to application-layer request data, enabling detailed inspection of attack payloads and user interactions. Under HTTPS, the same activity generates observable network traffic, but the application-layer contents are encrypted, shifting visibility toward metadata and connection-level behavior.

Feature	HTTP	HTTPS
Payload Visibility	Full	Not Visible
Suricata Detection	Stronger	Reduced
Zeek HTTP Logging	Visible in http.log	Not visible for encrypted requests
Zeek TLS Metadata	Not applicable	Visible in ssl.log
Flow Behavior	Visible	Visible
Attack Visibility	Direct	Indirect
Analyst Value	Payload and behavior	Metadata and behavior

Table 5.1: HTTP vs HTTPS visibility comparison

Table 5.1 summarizes the visibility differences observed during the experiment. The HTTP phase enabled direct inspection of application-layer requests, allowing both Suricata and Zeek to analyze traffic content and behavior. In contrast, the HTTPS phase limited analysis to

TLS session metadata and connection patterns, removing access to request payloads while preserving indicators such as session frequency, timing, and communication endpoints.

These findings are consistent with recent encrypted traffic research, which emphasizes that TLS-protected traffic can still reveal useful behavioral and fingerprinting features even when payload contents are hidden. Yu et al. describe TLS fingerprinting as a method for detecting encrypted malicious traffic by analyzing session characteristics, temporal behavior, and statistical features rather than decrypted content [6]. This aligns with the results of this project because HTTPS reduced direct inspection, but metadata and flow-level evidence remained available for analysis.

The results demonstrate that TLS encryption does not eliminate network visibility, but it fundamentally changes how detection must be performed. Suricata and Zeek fulfill complementary defensive roles: Suricata focuses on signature-based detection, which is strongest when payload data is available, while Zeek provides structured evidence of network behavior even when payload contents are hidden. Together, these tools demonstrate why encrypted environments require multiple forms of visibility.

The Burp Suite and Nginx configuration clarified an important distinction. Burp Suite was able to display HTTP request contents because it functioned as an intercepting proxy during controlled testing. Nginx handled HTTPS for the Juice Shop application using a self-signed TLS certificate, allowing traffic to be encrypted between the browser and server. From the perspective of network monitoring tools, the traffic was still HTTPS on port 443. Therefore, the visibility available to Burp Suite should not be confused with the visibility available to a passive network IDS.

In a real enterprise environment, web applications, mobile applications, authentication systems, and customer-facing services commonly operate over HTTPS. Encryption protects customer data confidentiality but simultaneously reduces the effectiveness of traditional network-based inspection methods. Organizations cannot rely only on payload inspection. They must combine IDS telemetry with authentication logs, application logs, proxy records, endpoint visibility, and behavioral monitoring to identify suspicious activity such as account abuse, automated login attempts, abnormal API usage, or suspicious encrypted sessions.

The practical implication is that encrypted environments require layered detection. A SOC analyst or cybersecurity analyst would use Suricata alerts to identify known signatures when available, Zeek logs to investigate connection behavior and TLS metadata, and application or authentication logs to confirm what occurred inside the encrypted session. This layered approach is necessary because no single tool provides complete visibility once TLS encryption is introduced.

Overall, the results supported the project hypothesis. Network-based detection tools experienced reduced effectiveness for application-layer attacks when payload inspection was removed by TLS encryption. However, visibility was not completely lost. Zeek continued to provide useful metadata and connection-level evidence, while Suricata remained useful primarily when traffic matched available rule-based indicators. These findings confirm that TLS encryption introduces a fundamental visibility tradeoff: it strengthens confidentiality while reducing the effectiveness of traditional payload-based intrusion detection methods.

Chapter 6

Contribution

This project contributes a practical, experimental evaluation of how TLS encryption impacts network-based intrusion detection visibility in cloud environments. By implementing a controlled AWS-based lab and executing identical attack scenarios over both HTTP and HTTPS, this study provides a direct comparison of how encryption affects real-world IDS tools such as Suricata and Zeek.

Unlike prior work that focuses primarily on theoretical models or large-scale datasets, this project demonstrates the visibility tradeoff through hands-on experimentation. The results highlight the limitations of signature-based detection under encryption and emphasize the continued value of metadata and behavioral analysis. This contribution supports the development of more effective detection strategies in modern encrypted network environments.

Conclusion and Future Work

The results of this project confirm that TLS encryption significantly reduces application-layer visibility for network-based intrusion detection systems. While HTTP traffic allowed full inspection of request contents and attack payloads, HTTPS traffic limited monitoring tools to encrypted session metadata and connection-level behavior. This demonstrates a clear security visibility tradeoff: encryption protects data confidentiality but reduces the effectiveness of traditional payload-based detection. Despite this limitation, detection is not eliminated. Tools such as Zeek continue to provide valuable insight through TLS metadata and behavioral patterns, while Suricata remains useful when traffic matches known signatures. These findings highlight the need for a layered detection approach in modern environments, where network telemetry must be combined with application logs, authentication data, and behavioral analysis. TLS encryption significantly reduces application-layer detection visibility while preserving limited metadata and behavioral insight. Suricata focuses on signature-based detection, while Zeek focuses on generating detailed network metadata for analysis. These tools complement each other rather than serve the same purpose.

Future work should explore the use of artificial intelligence and machine learning to improve detection in encrypted traffic environments. Research by Kim and Kim (2024) demonstrates that anomaly detection models using packet metadata and flow-based features can significantly improve detection accuracy without requiring payload inspection. Similarly, TLS fingerprinting techniques show that encrypted traffic can still reveal identifiable patterns through statistical and temporal characteristics. Building on this foundation, future implementations could integrate machine learning models, TLS fingerprinting methods, and real-time behavioral analytics to enhance detection capability in encrypted environments. These approaches represent a critical direction for modern cybersecurity, where encryption is ubiquitous and traditional inspection methods are no longer sufficient.

REFERENCES

- [1] M. Qutqut et al., “A comparative evaluation of Snort and Suricata for detecting data exfiltration tunnels in cloud environments,” *Journal of Cloud Computing*, 2024.
- [2] R. Zheng, J. Liu, L. Liu, S. Liao, K. Li, J. Wei, L. Li, and Z. Tian, “Two-layer detection framework with a high accuracy and efficiency for a malware family over the TLS protocol,” *PLOS ONE*, vol. 15, no. 5, 2020.
- [3] B. Anderson, “Detecting Encrypted Malware Traffic (Without Decryption),” Cisco Security Blog.
- [4] R. Antonello et al., “Deep packet inspection tools and techniques in commodity platforms: Challenges and trends,” *Journal of Network and Computer Applications*, 2012.
- [5] P. Zhang et al., “Detection and utilization of new-type encrypted network traffic in distributed scenarios,” *Engineering Applications of Artificial Intelligence*, 2023.
- [6] L. Yu, J. Tao, Y. Xu, W. Sun, and Z. Wang, “TLS fingerprint for encrypted malicious traffic detection with attributed graph kernel,” *Computer Networks*, vol. 247, p. 110475, June 2024, doi: 10.1016/j.comnet.2024.110475.
- [7] Kim, M.-G., & Kim, H. (2024). *Anomaly Detection in Imbalanced Encrypted Traffic with Few Packet Metadata-Based Feature Extraction*. *Computer Modeling in Engineering & Sciences*. <https://doi.org/10.32604/cmes.2024.051221>